

CHAPTER

02 파이썬 핵심 문법

CONTENTS

- 2.1 코랩 환경
- 2.2 변수
- 2.3 연산자
- 2.4 표준 입·출력
- 2.5 컬렉션 자료형
- 2.6 리스트 내포
- 2.7 선택문
- 2.8 반복문과 기타 제어문
- 2.9 함수

개념 잡는 데이터 분석

with 머신러닝

장은실, 오경선, 양숙희 지음



21 코랩 환경

코랩(Colab, 또는 Colaboratory)이란

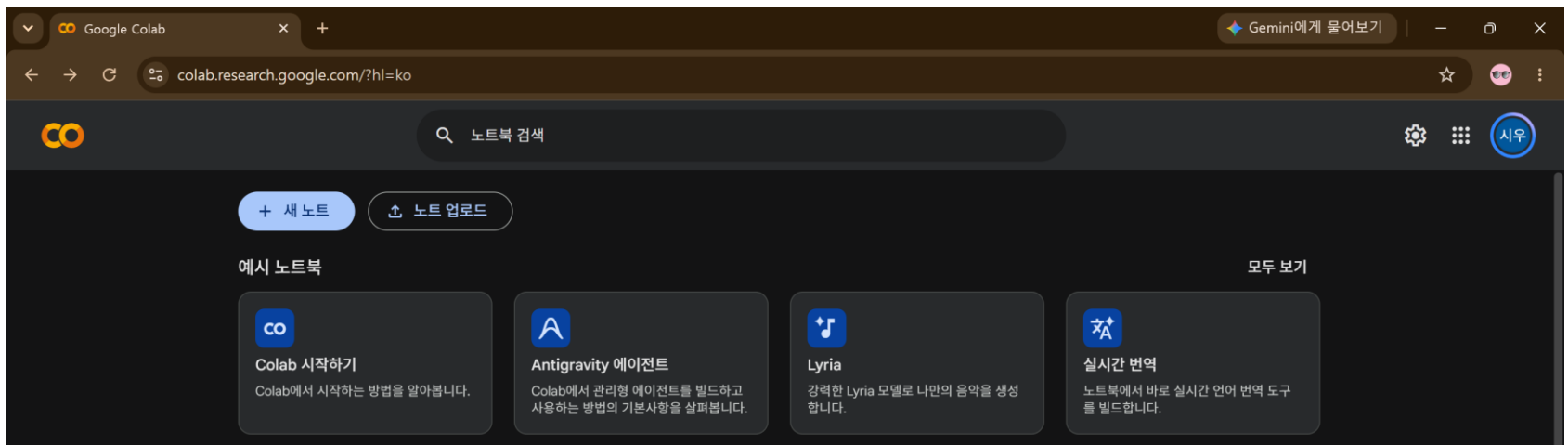
코랩

- 구글에서 교육과 과학 연구를 목적으로 개발한 도구
- 대중교통에서 스마트폰으로 코랩 노트북에 접속해서 텍스트와 파이썬 코드 작성 및 실행할 수 있다!
- 파이썬 코드 실행은 구글 클라우드 가상 서버에서 이루어진다.
- 별도의 파이썬 라이브러리를 설치하지 않아도 사용할 수 있다.
- 코랩에서 만든 파일은 구글 드라이브에 저장하고 불러올 수 있다.
- 구글 드라이브 문서와 같이 링크만으로 공유 및 협업 가능
- 단, 코드 실행 시 딜레이 존재

코랩 기본 사용법

준비물은 브라우저와 구글 계정뿐

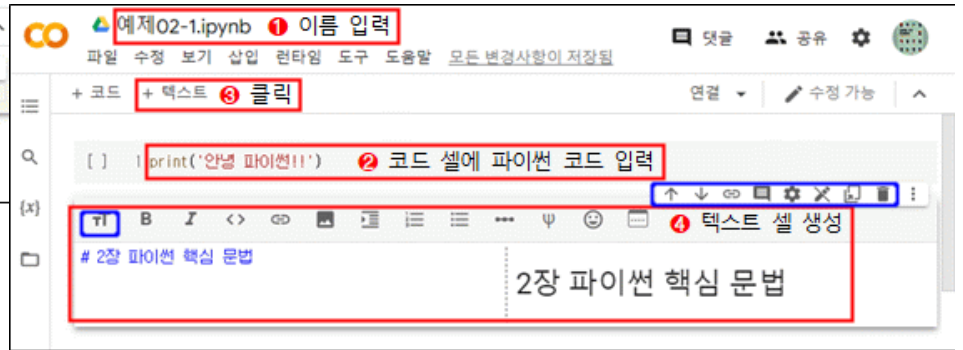
- 설치·설정 없이 브라우저만 있으면 바로 시작 (Chrome 권장)
- 주소창에 colab.research.google.com 입력
- 구글 계정으로 로그인 (또는 학교 계정 사용)
- 시작 화면에서 [새 노트] 클릭 → 노트북 생성 완료
- 만든 노트북은 내 구글 드라이브에 자동 저장



새 노트북 만들기과 화면 구성

노트북 화면 한눈에 보기

- 파일 이름(.ipynb)을 클릭해 변경 : 예) 1주차_실습
- 상단 메뉴 : 파일 / 수정 / 삽입 / 런타임 / 도구
- [+ 코드], [+ 텍스트] 버튼으로 셀을 추가한다
- 왼쪽 패널 : 목차, 파일, 코드 스니펫
- 오른쪽 위 RAM·디스크 표시는 현재 런타임 연결 상태



코드 셀 실행하기

실행은 재생(▶) 버튼 또는 Shift + Enter

- 코드 셀에 `print('안녕 파이썬')` 을 입력하고 Shift + Enter
- 실행 결과는 셀 바로 아래에 표시된다
- [1], [2] 같은 번호는 실행된 순서를 뜻한다
- Ctrl + Enter : 실행 후 현재 셀 유지 / Alt + Enter : 실행 후 새 셀로 이동
- 실습) $2 + 3 * 4$, $7 / 2$ 의 결과를 직접 확인해 보자



코랩 기본 사용법

기타 기능

- 다양한 단축키를 확인 및 설정



텍스트 셀과 마크다운

코드와 설명을 함께 남기는 학습 노트

- [+ 텍스트] 버튼으로 설명.필기용 셀을 추가한다.
- # 제목1, ## 제목2 : 글자 크기와 구조를 만든다.
- **굵게**, *기울임*, - 목록, 1. 번호 목록
- $y = ax + b$ 처럼 수식도 입력할 수 있다.
- 실습) 오늘 배운 내용을 텍스트 셀에 3줄로 정리하기

장제목

절 제목

항목

어디까지

글쓰기 연습 글을 써보세요.

기울임 글씨 기울임 글씨를 작성했어요.

- 학교
- 교통

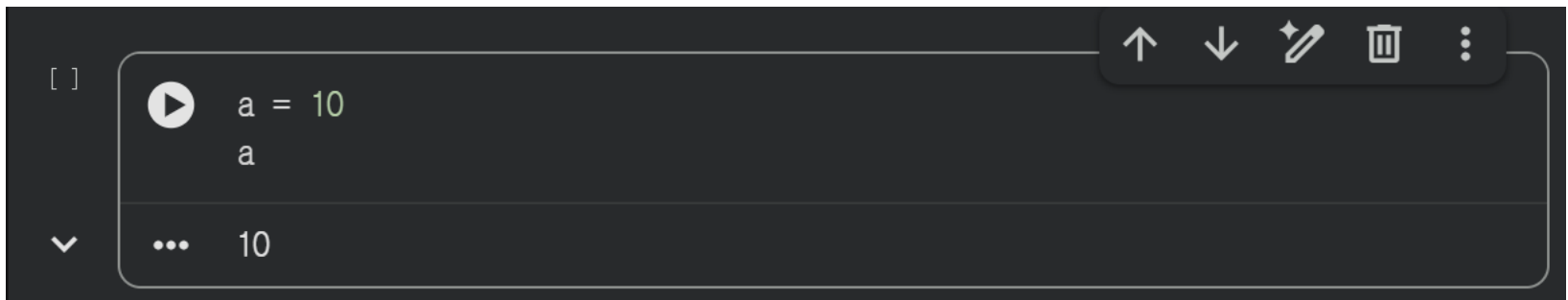
1. 첫번째
2. 세번째

$$y = ax + b$$

셀 관리와 실행 순서

모든 셀은 같은 메모리를 공유한다

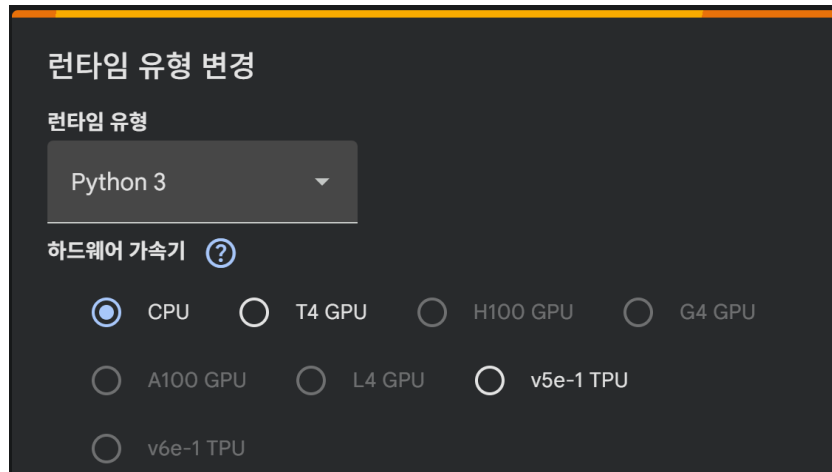
- 셀 위·아래 이동, 복사, 삭제(휴지통) 버튼 활용
- 앞 셀에서 만든 변수는 뒤 셀에서 계속 사용할 수 있다
- 앞 셀을 실행하지 않으면 NameError 오류가 발생한다
- 순서가 헷갈리면 [런타임] → [모두 실행]
- 노트북은 위에서 아래로 읽히도록 정리하는 습관을 들이자



런타임 관리

코드를 실행해 주는 구글 클라우드 컴퓨터

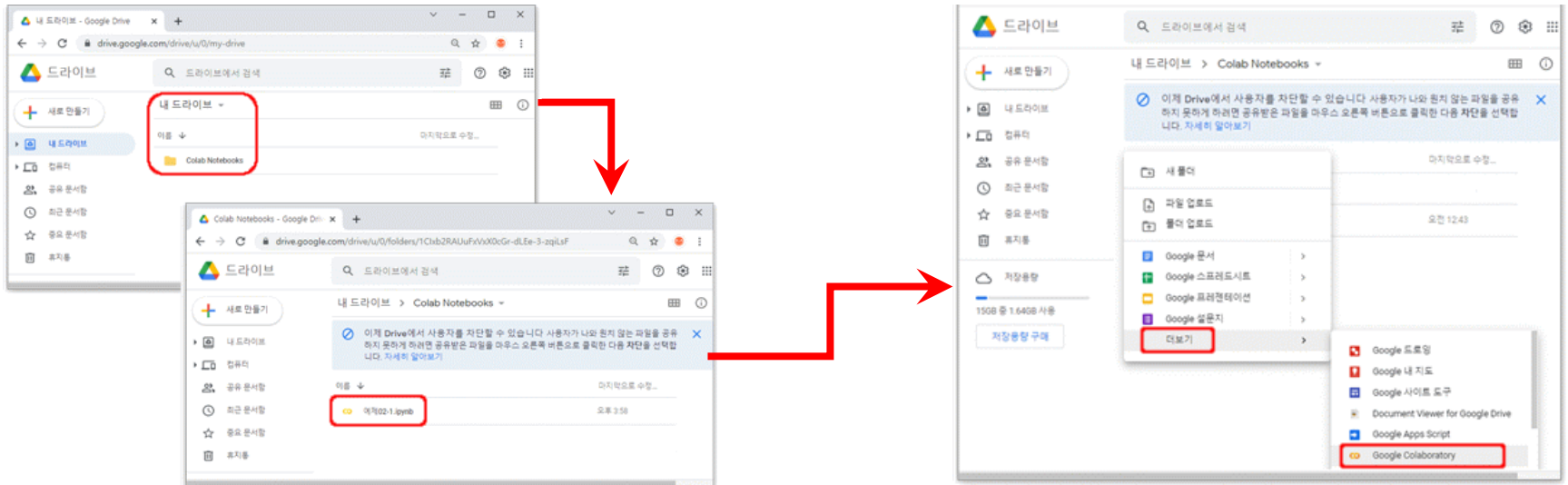
- [런타임] → [세션 다시 시작] : 변수와 메모리를 초기화
- [런타임] → [런타임 유형 변경] : CPU / GPU / TPU 선택
- 약 90분간 조작이 없으면 연결이 끊어진다
- 최대 연속 사용 시간은 약 12시간 (무료 기준)
- 연결이 끊기면 코드는 남지만 변수는 사라진다 → 다시 실행



파일 다루기와 구글 드라이브 연결

데이터는 드라이브에 두는 것이 안전하다

- 왼쪽 [파일] 패널에 끌어 놓기 = 임시 업로드 (세션 종료 시 삭제)
- 드라이브 연결 : from google.colab import drive
- `drive.mount('/content/drive')` 실행 후 권한 승인
- 연결된 경로 : `/content/drive/MyDrive/` 아래 내 파일들
- 예) `pd.read_csv('/content/drive/MyDrive/data.csv')`



라이브러리 사용하기

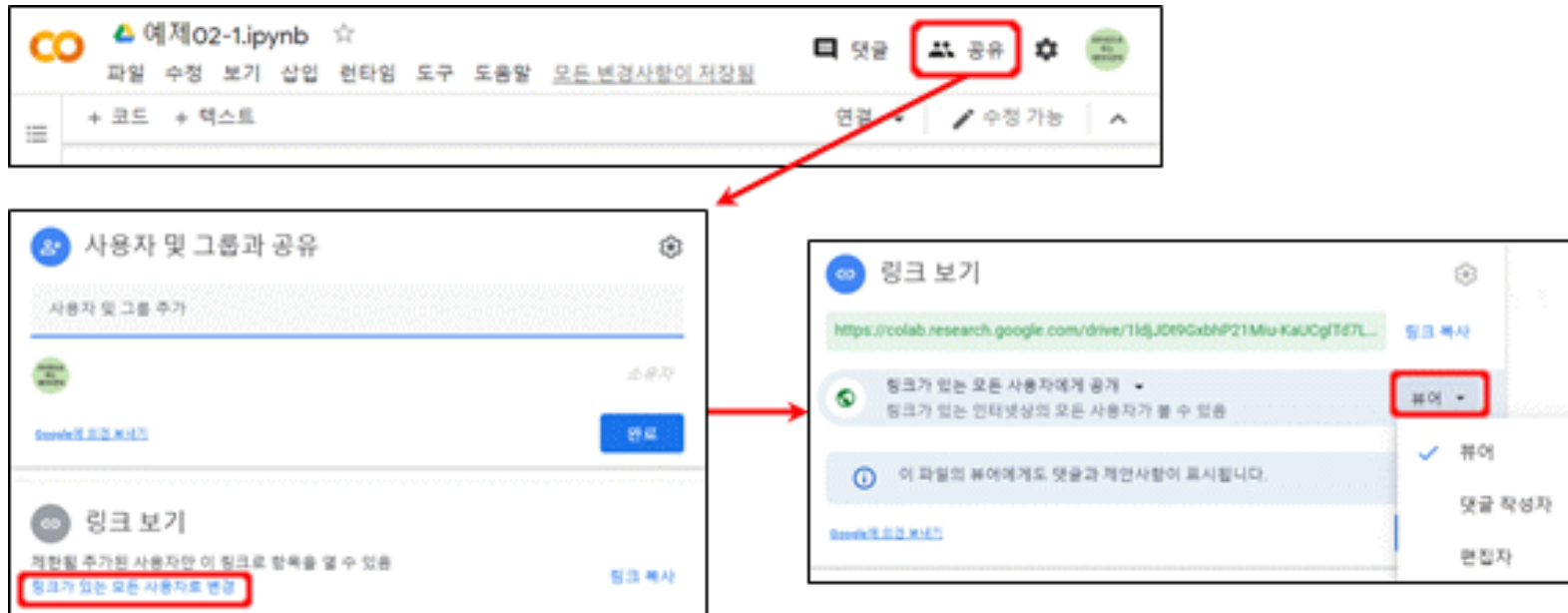
대부분은 이미 설치되어 있다

- pandas, numpy, matplotlib 등은 기본으로 제공된다
- 사용 전에 `import pandas as pd` 처럼 불러오기
- 없는 라이브러리는 `!pip install 라이브러리명` 으로 설치
- 맨 앞의 `!` 는 리눅스 명령 실행을 뜻한다 (`!ls`, `!pwd`)
- 설치한 라이브러리는 세션이 끝나면 다시 설치해야 한다

코랩 기본 사용법

기타 기능

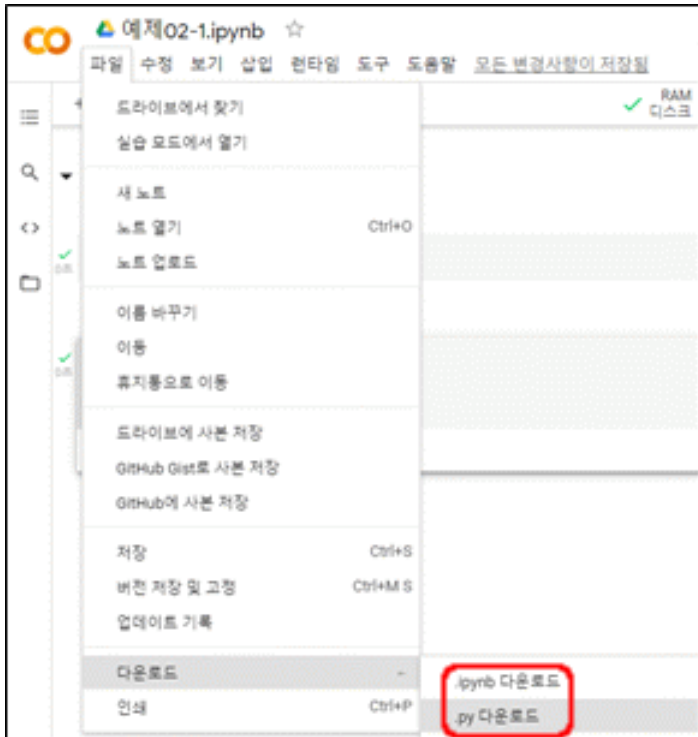
- 공유 기능



코랩 기본 사용법

기타 기능

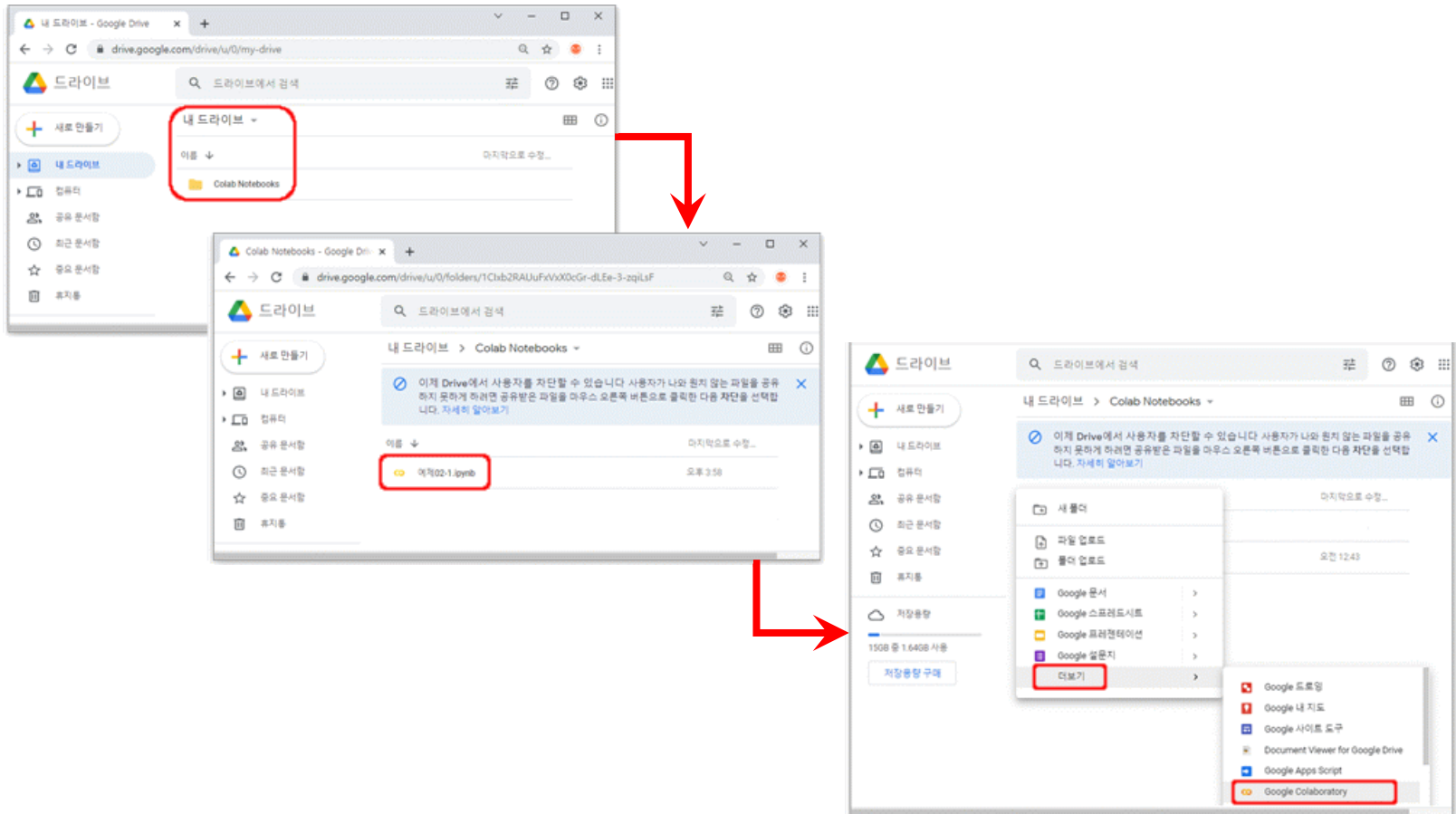
- .py 다운로드



코랩 기본 사용법

기타 기능

- 구글 드라이브에서 확인 및 코랩 불러오기

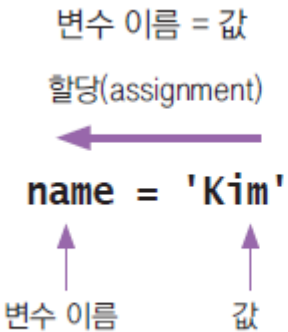


2.2 변수

변수 특징

- 일시적으로 자료를 저장하는 공간이며, 변수에 저장된 값은 변할 수 있다.
- 변수에는 숫자, 문자열 등 모든 자료형을 저장할 수 있고, 다른 변수의 값도 저장할 수 있다.
- 변수는 사용되기 전에 반드시 할당되어 있어야 한다.

변수 생성 방법

구조	 The diagram illustrates the structure of a variable assignment statement. At the top, it says '변수 이름 = 값' (Variable name = value). Below this, '할당(assignment)' (Assignment) is written. A horizontal arrow points from the right towards the assignment operator. Below the arrow, the code 'name = 'Kim'' is shown. Two vertical arrows point upwards: one from 'name' to the variable name label '변수 이름' (Variable name), and another from ''Kim'' to the value label '값' (Value).	- name은 변수명, ‘ ’ 안에 있는 Kim은 데이터, 즉 값이라는 뜻이다. - 파이썬에서 변수에 문자열 데이터를 할당할 때는 따옴표(‘ ’ 또는 “ ”) 안에 써야 한다.
----	---	--

변수 생성 및 규칙

- 규칙1 : 영문자, 숫자, 밑줄 문자(_)로 구성하며, 첫 글자는 반드시 영문자/밑줄 문자(_)로 시작한다. 밑줄 문자(_)를 제외한 특수문자는 사용할 수 없다.
- 규칙2 : 공백이 들어가면 안 된다.
- 규칙3 : 대문자와 소문자를 구별한다. Value와 value는 다른 변수이다.
- 규칙4 : 파이썬에서 다른 용도로 사용되는 예약어는 변수명으로 사용할 수 없다.

유효한 변수명	유효하지 않은 변수명
age	2name # 숫자로 시작할 수 없음
_count # 밑줄 문자로 시작 가능	stu no # 공백을 포함할 수 없음
value3 # 맨 처음이 아니면 숫자 포함 가능	price@ # @ 특수문자 사용 안 됨
stu_no # 밑줄 문자는 중간에 포함 가능	if # 예약어 안 됨

변수 생성 및 규칙

 **Note** 파이썬의 주석 처리

- 한 줄 주석 처리: #
- 여러 줄 주석 처리: ''' ~ ''' / """ ~ """

 **Note** 기본 자료형의 종류

자료형	의미	예시
int	integer, 정수	x=100
float	float, 부동 소수점	x=95.7
str	string, 문자열	x='Lee' x="Lee"
bool	boolean, 부울	x=True x=False

2.3 연산자

연산자

산술 연산자

- 파이썬에서 사용하는 기본적인 산술 연산자는 다음과 같다.

연산자	의미	연산자	의미
+	더하기	-	빼기
*	곱하기	/	나누기
%	나머지	**	거듭제곱
//	몫		

연산자

산술 연산자

산술 연산자: x=10, y=20일 때

1 x=10 2 y=20 3 print(x+y)	1 x=10 2 y=20 3 print(x/y)
30	0.5
1 print(x-y)	1 print(x%y)
-10	10
1 print(x*y)	1 print(x**2)
200	100
	1 print(x//y)
	0

대입 및 복합 대입 연산자

- 연산자 짧게 사용하기
- 복합 대입 연산자 : 산술 연산자 + 대입 연산자

연산자	의미	예시
=	왼쪽 변수에 오른쪽 값을 할당	x=10
+=	왼쪽 변수에 오른쪽 값을 더하고 결과를 왼쪽 변수에 할당	x+=10 (x=x+10)
-=	왼쪽 변수에 오른쪽 값을 빼고 결과를 왼쪽 변수에 할당	x-=10 (x=x-10)
=	왼쪽 변수에 오른쪽 값을 곱하고 결과를 왼쪽 변수에 할당	x=10 (x=x*10)
/=	왼쪽 변수에 오른쪽 값을 나눈 결과를 왼쪽 변수에 할당	x/=10 (x=x/10)
%=	왼쪽 변수에 오른쪽 값을 나눈 나머지의 결과를 왼쪽 변수에 할당	x%=10 (x=x%10)
=	왼쪽 변수에 오른쪽 값만큼 거듭제곱을 하고 결과를 왼쪽 변수에 할당	x=2 (x=x**2)
//=	왼쪽 변수에서 오른쪽 값을 나눈 몫의 결과를 왼쪽 변수에 할당	x//=10 (x=x//10)

비교(관계) 연산자

- 파이썬에서 사용할 수 있는 관계(비교)연산자는 다음과 같다.

연산자	의미
==	값이 동일하다.
!=	값이 동일하지 않다.
>	왼쪽 값이 오른쪽 값보다 크다.
>=	왼쪽 값이 오른쪽 값보다 크거나 동일하다.
<	왼쪽 값이 오른쪽 값보다 작다.
<=	왼쪽 값이 오른쪽 값보다 작거나 동일하다.

연산자

논리 연산자

- 파이썬에서 사용할 수 있는 논리연산자는 다음과 같다.

연산자	의미
and	논리 AND 연산. 둘 다 참일 때만 참
or	논리 OR 연산. 둘 중 하나만 참이여도 참
not	논리 NOT 연산. 논리 상태를 반전

연산자

문자열 연산자

- 파이썬에서는 문자열을 더하거나 곱할 수 있다.

문자열 더하기: +	문자열 곱하기: *
<pre>1 start='파이썬' 2 end='짱!' 3 start+end</pre>	<pre>1 word='파이썬' 2 word*5</pre>
'파이썬 짱!'	'파이썬 파이썬 파이썬 파이썬 파이썬'

Note

*는 산술 연산자의 숫자 곱하기와는 의미가 다르다. 위 소스 코드에서 `word*5` 문장은 `word`를 5번 반복하라는 뜻이다. 즉 문자열에서 * 연산자는 문자열 반복을 의미한다.

2.4 표준 입·출력

표준 입·출력

표준 입력

- input() 함수를 이용하여 자료를 입력받고 저장한다.

입력 자료 저장

```
1 name = input('이름 입력: ')
2 print(name)
```

실행 결과

이름 입력: 이순신
이순신

정수형으로 입력된 자료

```
1 x = int(input('숫자1 입력: '))
2 y = int(input('숫자2 입력: '))
3 print(x+y)
```

실행 결과

숫자1 입력: 20
숫자2 입력: 30
50

실수형으로 입력된 자료

```
1 z = float(input('숫자 입력: '))
2 print(z)
```

실행 결과

숫자 입력: 7.5
7.5

표준 입·출력

표준 출력

- print() 함수를 이용하여 자료를 출력한다.
- end 옵션을 이용하면 줄 바꿈을 제어할 수 있다.

줄 바꿈 출력	end 옵션을 이용한 줄 바꿈 제어
<pre>1 print('파이썬') 2 print('짱')</pre>	<pre>1 print('파이썬', end=' ') 2 print('짱')</pre>
실행 결과 파이썬 짱	실행 결과 파이썬 짱

표준 입·출력

표준 출력

- 콤마(,)를 이용한 출력

☞ 콤마(,)로 구분하여 출력

```
1 name = input('이름 입력: ')\n2 print('입력한 이름은', name)
```

실행 결과

이름 입력: 이순신
입력한 이름은 이순신

- % 형식지정자를 이용한 출력

☞ %s 형식지정자를 이용하여 출력

```
1 name = input('이름 입력: ')\n2 print('입력한 이름은 %s' % name)
```

실행 결과

이름 입력: 이순신
입력한 이름은 이순신

표준 입·출력

표준 출력

- % 형식지정자를 이용한 출력

☰ %d 형식지정자를 이용하여 출력

```
1 x = int(input('숫자1 입력: '))
2 y = int(input('숫자2 입력: '))
3 print('%d + %d = %d' % (x, y, x+y))
```

실행 결과

```
숫자1 입력: 20
숫자2 입력: 30
20 + 30 = 50
```

☰ %f 형식지정자를 이용하여 출력

```
1 x = int(input('숫자1 입력: '))
2 y = int(input('숫자2 입력: '))
3 print('%d / %d = %f' % (x, y, x/y))
```

실행 결과

```
숫자1 입력: 5
숫자2 입력: 2
5 / 2 = 2.500000
```

표준 입·출력

표준 출력

- format() 함수를 이용한 출력

format() 함수를 이용하여 출력

```
1 x = int(input('숫자1 입력: '))
2 y = int(input('숫자2 입력: '))
3 print('{0} * {1} = {2}'.format(x, y, x*y))
```

실행 결과

```
숫자1 입력: 7
숫자2 입력: 3
7 * 3 = 21
```

중괄호 순서를 변경하고 자료를 반복하여 출력

```
1 print('{1} {1} 작은 {0}'.format('별', '반짝'))
```

실행 결과

```
반짝 반짝 작은 별
```

```
print('{1} {1} 작은 {0}'.format('별', '반짝'))
```

표준 입·출력

표준 출력

- f-string 포매팅을 이용한 출력

☞ f-string 포매팅을 이용하여 출력

```
1 x = int(input('숫자1 입력: '))  
2 y = int(input('숫자2 입력: '))  
3 print(f'{x} - {y} = {x-y}')
```

실행 결과

```
숫자1 입력: 30  
숫자2 입력: 20  
30 - 20 = 10
```

2.5 컬렉션 자료형

컬렉션 자료형

컬렉션 자료형 개념과 필요성

- 한 번에 여러 개의 값을 저장하거나 여러 가지 데이터들을 관계있는 그룹으로 관리하는 것

표 2-6 컬렉션 자료형의 종류

컬렉션 자료형 종류	생성 방법	컬렉션 자료형 종류	생성 방법
리스트(list)	[]	딕셔너리(dictionary)	{키:값}
튜플(tuple)	()	세트(set)	{ }

컬렉션 자료형

리스트

- 하나의 리스트 변수를 이용하여 여러 개의 자료를 효율적으로 저장하고 처리함

리스트 변수 사용

```
1 x = [0, 0, 0, 0]
2 x[0] = int(input('점수1: '))
3 x[1] = int(input('점수2: '))
4 x[2] = int(input('점수3: '))
5 x[3] = int(input('점수4: '))
6
7 hap=x[0]+x[1]+x[2]+x[3]
8
9 print('합계:',hap)
```

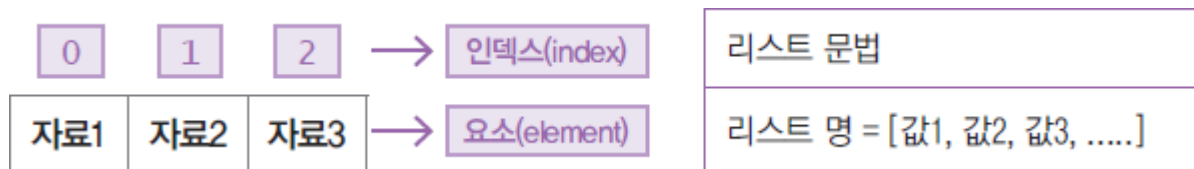
실행 결과

```
점수1: 10
점수2: 20
점수3: 30
점수4: 40
합계: 100
```

컬렉션 자료형

리스트 문법

- 리스트 대괄호([]) 안에 서로 다른 자료형의 값을 콤마(,)로 구분해 하나 이상 저장할 수 있는 컬렉션 자료형
- 대괄호([])에 넣는 자료를 요소(element)라고 부르며, 요소들은 순서를 가지고 있고 인덱스를 사용하여 참조할 수 있음



리스트 인덱싱(Indexing) 및 슬라이싱(Slicing)

- 리스트의 첫 번째 요소 위치는 0에서 시작하여 리스트명[0], 리스트명[1], ... 으로 표기하고, 역 인덱싱은 리스트명[-1], 리스트명[-2], ... 으로 표기하여 마지막 요소부터 접근할 수 있음
- 슬라이싱은 범위를 지정하여 리스트 명[시작:끝]과 같은 형식을 사용하고 시작 요소부터 (끝-1) 인덱스에 있는 요소까지 선택함

컬렉션 자료형

리스트 인덱싱(Indexing) 및 슬라이싱(Slicing)

리스트 인덱스 구조 및 사용하기

0	1	2	3	← 인덱스
15	25	35	45	
-4	-3	-2	-1	← 역 인덱스

```
1 x = [15,25,35,45]
2 print(x)
```

```
[15, 25, 35, 45]
```

```
1 x[0] # 리스트의 첫 번째 값을 가져온다.
```

```
15
```

```
1 x[0]+x[1]
```

```
40
```

```
1 x[-1] # 리스트의 마지막 값을 가져온다.
```

```
45
```

```
1 x[4]
```

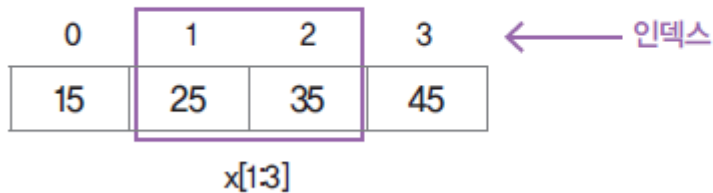
```
-----
IndexError                                Traceback (most recent call last)
<ipython-input-18-ed40d1f49c42> in <module>()
----> 1 x[4]
```

```
IndexError: list index out of range
```


컬렉션 자료형

리스트 인덱싱(Indexing) 및 슬라이싱(Slicing)

리스트 슬라이싱 구조



리스트 슬라이싱 사용하기

1 x = [10, 20, 30, [40, 50]]	1 x[3]
2 x[0:2]	
[10, 20]	[40, 50]
1 x[1:]	1 x[3][0] # 이중 리스트 슬라이싱
[20, 30, [40, 50]]	40
1 x[:2]	1 x[3][1] # 이중 리스트 슬라이싱
[10, 20]	50
1 x[-1]	
[40, 50]	

컬렉션 자료형

리스트 값 변경

리스트 값 변경하기

```
1 a=[10,20,30]
2 print(a)
```

[10, 20, 30]

```
1 a[1:3]=[90,100] # 연속된 범위의 값도 변경 가능하다.
2 print(a)
```

[100, 90, 100]

```
1 a[0]=100
2 print(a)
```

[100, 20, 30]

리스트 제어 함수

- 리스트를 제어할 수 있는 유용한 제어 함수들이 있는데, 변수 이름 뒤에 마침표(.)를 붙인 다음 함수 이름을 사용함

리스트 제어 함수의 종류

함수	설명	사용법
append()	리스트에 요소를 마지막 위치에 새로 추가	리스트.append(값)
insert()	리스트의 해당 위치에 요소를 새로 삽입	리스트.insert(위치, 값)
sort()	오름차순 정렬 내림차순 정렬	리스트.sort() 리스트.sort(reverse=True)
reverse()	현재의 리스트를 그대로 거꾸로 뒤집기	리스트.reverse()
pop()	리스트 제일 뒤의 항목을 빼내고 빼낸 항목은 삭제	리스트.pop()
	제거할 위치에 있는 요소를 제거	리스트.pop(위치)
remove()	해당 요소를 찾아 삭제	리스트.remove(삭제할 값)
count()	해당 요소의 개수를 반환	리스트.count(찾을 값)
index()	리스트에 위치 값이 있으면 위치값을 반환	리스트.index(값)
len()	리스트 총 요소 개수를 반환	len(리스트)

컬렉션 자료형

튜플

- 튜플은 한번 저장된 값은 수정할 수 없는 자료형으로 읽기 전용의 데이터를 저장할 때 유용하게 사용함

튜플 초기화

```
1 x = ()
2 print(x)
```

()

```
1 x = tuple()
2 print(x)
```

()

```
1 x = tuple(range(10))
2 print(x)
```

(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

```
1 x = (1,2,3)
2 print(x)
```

(1, 2, 3)

```
1 x = 1,2
2 print(x)
```

(1, 2)

```
1 x = (10,20,(30,40))
2 print(x)
```

(10, 20, (30, 40))

```
1 x = ['월','화','수','목','금','토','일']
2 x = tuple(x) # 리스트를 튜플로 자료형 변환
3 print(x)
```

('월', '화', '수', '목', '금', '토', '일')

컬렉션 자료형

튜플 데이터 변경은 오류 발생

튜플 데이터 변경

```
1 x = (10,20,30)
2 x[0] = 100
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-65-b48a500f82af> in <module>()
      1 x = (10,20,30)
----> 2 x[0]=100
```

```
TypeError: 'tuple' object does not support item assignment
```

튜플의 인덱싱 및 슬라이싱

튜플 인덱싱과 슬라이싱

```
1 x = (10,20,30,40)
2 x[0]
```

10

```
1 x[-1]
```

40

```
1 x[1:]
```

(20, 30, 40)

```
1 x[:2]
```

(10, 20)

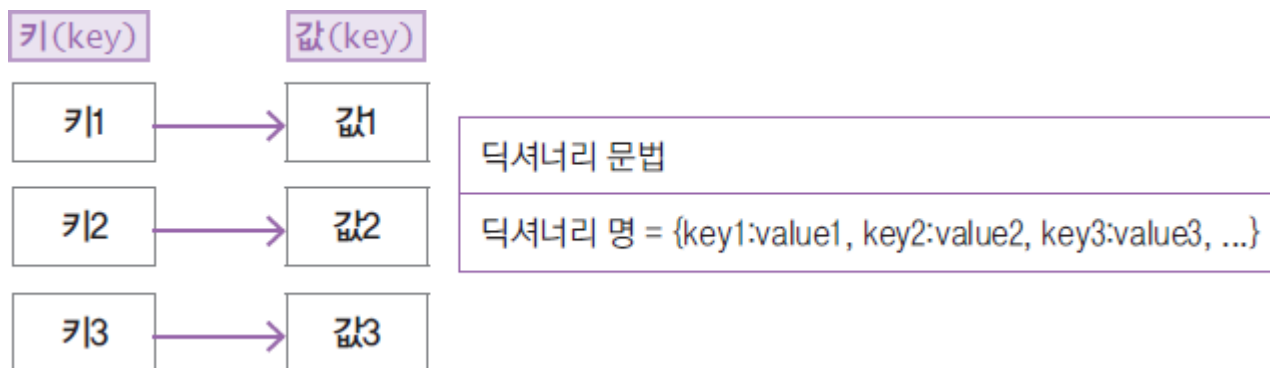
컬렉션 자료형

딕셔너리

- 딕셔너리는 순서가 없는 컬렉션 자료형으로 각각의 요소는 key:value 형태로 저장
- 딕셔너리는 리스트나 튜플처럼 인덱스에 의해 해당 요소를 찾지 않으며, key를 통해 value를 얻음

딕셔너리 문법

- {} 안에 키:값 형식의 항목을 콤마(,)로 구분해 하나 이상 저장할 수 있는 컬렉션 자료형



컬렉션 자료형

딕셔너리 요소에 접근

☰ 딕셔너리 요소 접근

```
1 menu={'김밥':3000,'라면':5000}  
2 menu['김밥']
```

3000

```
1 menu['라면']
```

5000

```
1 menu['순대']
```

```
1 menu['라면']=5500
```

```
2 menu['라면']
```

5500

```
1 menu['어묵']=1500
```

```
2 print(menu)
```

{'김밥': 3000, '라면': 5500, '어묵': 1500}

```
-----  
KeyError                                Traceback (most recent call last)  
<ipython-input-82-426a2e6cdc0a> in <module>()  
----> 1 menu['순대']
```

KeyError: '순대'

컬렉션 자료형

딕셔너리 제어 함수

함수	설명	사용법
<code>get()</code>	항목 접근하기	<code>딕셔너리.get(key)</code>
<code>pop()</code>	항목 꺼내고 삭제하기	<code>딕셔너리.pop(key)</code>
<code>del()</code>	항목 삭제하기	<code>del(딕셔너리[key])</code>
<code>items()</code>	딕셔너리에 저장된 모든 키와 값 반환	<code>딕셔너리.items()</code>
<code>keys()</code>	딕셔너리에 저장된 모든 키 반환	<code>딕셔너리.keys()</code>
<code>values()</code>	딕셔너리에 저장된 모든 값 반환	<code>딕셔너리.values()</code>

컬렉션 자료형

세트

- 세트는 집합에 관련된 것을 쉽게 처리하기 위해 만든 자료형으로 중복을 허용하지 않는 컬렉션 자료형
- 세트 자료형은 순서가 없기 때문에 인덱싱으로 값을 얻을 수 없음

세트 초기화

```
1 x = {10,20,30}
```

```
2 print(x)
```

```
{10, 20, 30}
```

```
1 y = set([10,20,30])
```

```
2 print(y)
```

```
{10, 20, 30}
```

```
1 x = {10,20,30,20}
```

```
2 print(x)
```

```
{10, 20, 30}
```

컬렉션 자료형

세트 연산

세트 합집합, 교집합 차집합

```
1 x = {10,20,30}
2 y = {20,40}
3 x & y          # 교집합
```

{20}

```
1 x.intersection(y) # 교집합
```

{20}

```
1 x | y          # 합집합
```

{20, 40, 10, 30}

```
1 x.union(y)     # 합집합
```

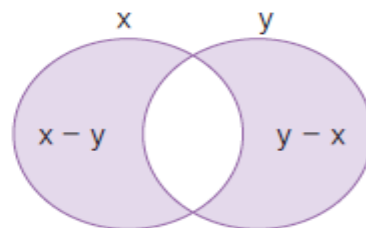
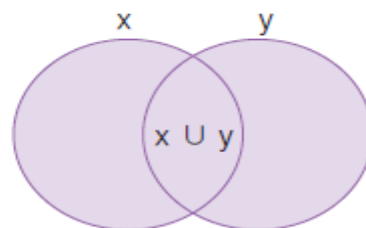
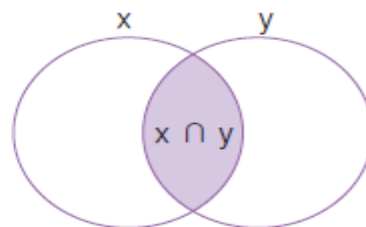
{20, 40, 10, 30}

```
1 x - y          # 차집합
```

{10, 30}

```
1 x.difference(y) # 차집합
```

{10, 30}



컬렉션 자료형

세트 제어 함수

함수	설명	사용법
<code>add()</code>	값 1개 추가	<code>세트.add(값)</code>
<code>update()</code>	값 여러 개 추가	<code>세트.update(값)</code>
<code>remove()</code>	특정 값 삭제	<code>세트.remove(값)</code>

2.6 리스트 내포

리스트 내포 개념

- 리스트 내포(List Comprehension)는 원하는 자료들을 조회 또는 추출하여 리스트로 변환하는 표현식
- 리스트 내포는 조건식을 이용하여 같은 연산을 전체 항목이나 일부 항목에 적용할 수 있음
- 리스트 내포를 이용하여 항목들을 대문자나 소문자로 바꾸는 등의 작업을 처리할 수 있음

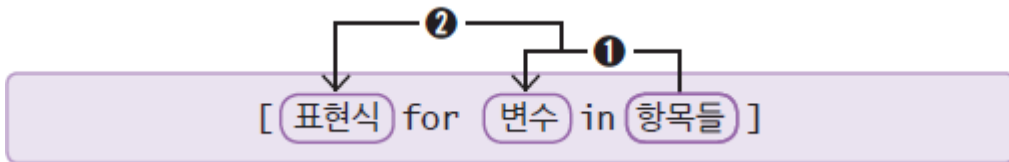
리스트 내포 처리 과정

- ① → ② → ③에서 ①로 이동하여 반복 수행, 더이상 꺼내 올 항목이 없으면 종료함
- ① 항목들에서 순차적으로 하나씩 꺼내온다.
- ② 조건식을 적용하여 해당 조건에 맞는 항목은 추출하고 조건에 맞지 않으면 무시한다.
- ③ 위에서 추출한 항목은 리스트에 추가한다.

리스트 내포

기본적인 리스트 내포 형식

- 항목들에서 순차적으로 꺼내 온 자료는 변수에 저장한 후 표현식에 적용하여 리스트에 추가함

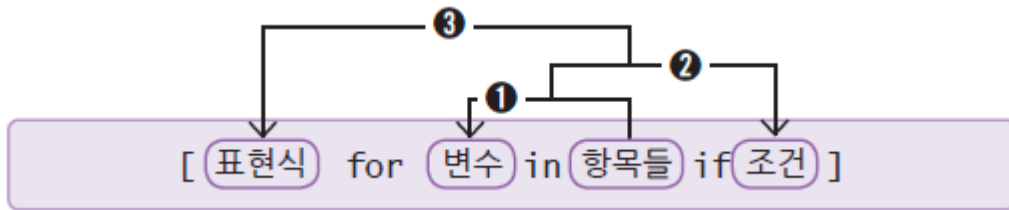


<pre>1 num = [-20,-10,0,10,20] 2 [x for x in num]</pre>	<pre>1 num = [-20,-10,0,10,20] 2 [x/5 for x in num]</pre>
<pre>[-20, -10, 0, 10, 20]</pre>	<pre>[-4.0, -2.0, 0.0, 2.0, 4.0]</pre>
<pre>1 num = [-20,-10,0,10,20] 2 [x+5 for x in num]</pre>	<pre>1 num = [-20,-10,0,10,20] 2 [x**2 for x in num]</pre>
<pre>[-15, -5, 5, 15, 25]</pre>	<pre>[400, 100, 0, 100, 400]</pre>

리스트 내포

조건 있는 리스트 내포 형식

- 항목들에서 순차적으로 꺼내온 자료는 변수에 저장한 후 조건에 맞는 항목만 추출한 다음 표현식에 적용하여 리스트에 추가함



1 num = [-20,-10,0,10,20] 2 [1/x for x in num if x>0]	1 num = [-20,-10,0,10,20] 2 [1/x for x in num if x!=0]
[0.1, 0.05]	[-0.05, -0.1, 0.1, 0.05]

2.7 선택문

선택문 개요

- 파이썬 선택문의 형태에는 if, if~else, if~elif, if~elif~else, if~in~else 등이 있으며, 파이썬 선택문의 규칙은 다음과 같다.
- ① if 문의 조건식들은 논리 연산이 가능한 문장으로 참·거짓 판별이 가능하도록 작성해야 한다.
- ② if 문의 조건식 끝에는 항상 :(콜론)이 있어야 한다.
- ③ 실행 코드는 반드시 공백(스페이스바 또는 Tab)으로 들여쓰기(indent)를 하여 if 문에 포함되는(종속) 코드로 작성해야 한다.

조건식

- 선택문을 표현하기 위해 필요한 조건식은 비교 연산자와 논리 연산자를 이용하여 표현하는데, 조건식을 만족하면 True, 만족하지 않으면 False의 결과 값을 가지게 됨

조건식의 예

조건식	설명
<code>total >= 100</code>	<code>total</code> 변수에 저장된 자료가 100 이상이면 만족하는 조건식
<code>start+end < 120</code>	<code>start</code> 변수와 <code>end</code> 변수를 더한 값이 120 미만이면 만족하는 조건식
<code>6 <= age < 60</code>	<code>age</code> 변수에 저장된 자료가 6 이상, 60 미만이면 만족하는 조건식
<code>num % 3 == 0</code>	<code>num</code> 변수에 저장된 자료를 3으로 나눈 나머지가 0이면 만족하는 조건식
<code>first != second</code>	<code>first</code> 변수와 <code>second</code> 변수에 저장된 자료가 다르면 만족하는 조건식
<code>eng>=80 and math>=80</code>	<code>eng</code> 변수에 저장된 자료가 80 이상이고 <code>math</code> 변수에 저장된 자료가 80 이상이면 만족하는 조건식
<code>weather=='여름' or weather=='겨울'</code>	<code>weather</code> 변수에 저장된 자료가 '여름'이거나 <code>weather</code> 변수에 저장된 자료가 '겨울'이면 만족하는 조건식
<code>not(flag==0)</code>	<code>flag</code> 변수에 저장된 자료가 0이 아니면 만족하는 조건식

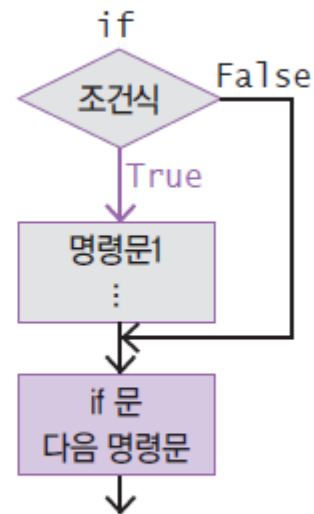
선택문

if 문

기본 문법

```
if 조건식:  
    (공백) 명령문1  
    ⋮  
if 문 다음 명령문
```

순서도



설명 if 문의 조건식 결과가 True라면 명령문1, ... 을 실행하고, 조건식의 결과가 False라면 명령문1, ... 을 건너뛰고 if 문 다음 명령문을 실행한다.

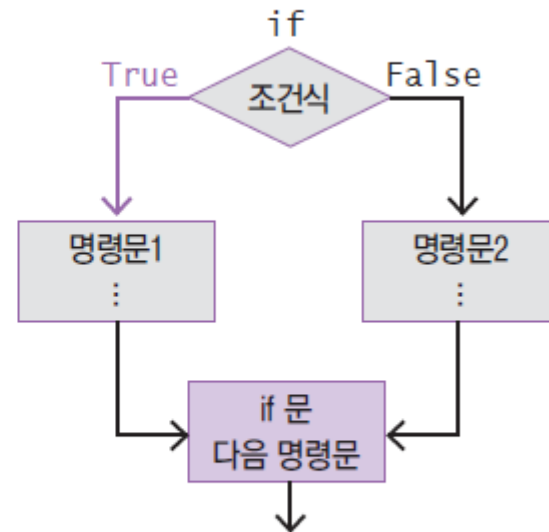
선택문

if~else 문

기본 문법

```
if 조건식:  
    (공백) 명령문1  
    ⋮  
else:  
    (공백) 명령문2  
    ⋮  
if 문 다음 명령문
```

순서도



설명

if 문의 조건식 결과가 True라면 명령문1, ... 을 실행하고 if 문 다음 명령문을 실행한다.
if 문의 조건식 결과가 False라면 명령문2, ... 를 실행하고 if 문 다음 명령문을 실행한다.

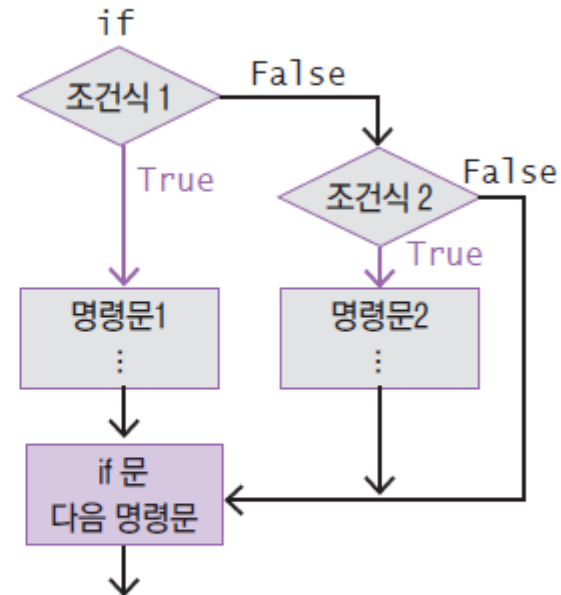
선택문

if~elif 문

기본 문법

```
if 조건식1:  
    (공백) 명령문1  
    :  
elif 조건식2:  
    (공백) 명령문2  
    :  
if 문 다음 명령문
```

순서도



설명 if 문의 조건식1의 결과가 True라면 명령문1, ... 을 실행하고, False라면 조건식2를 확인하여 결과가 True라면 명령문2, ... 를 실행하고, False라면 건너뛰고 if 문 다음 명령문을 실행한다.

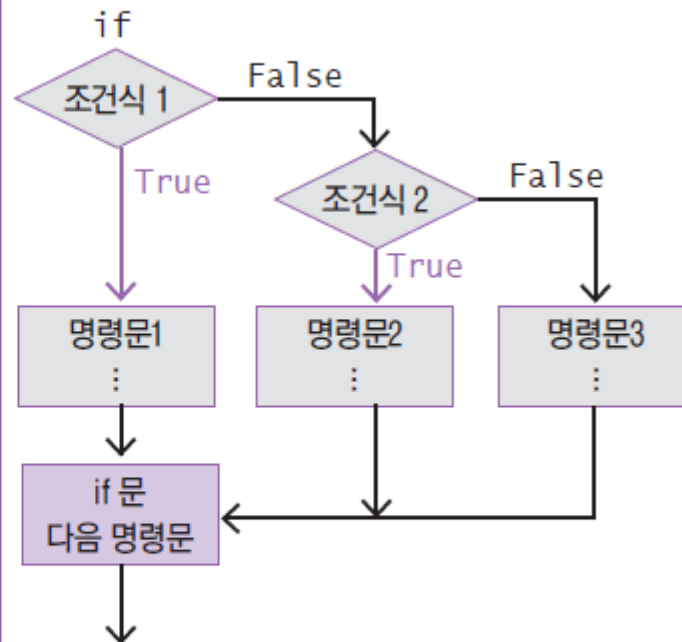
선택문

if~elif~else 문

기본 문법

```
if 조건식1:  
    (공백) 명령문1  
    :  
elif 조건식2:  
    (공백) 명령문2  
    :  
else:  
    (공백) 명령문3  
    :  
if 문 다음 명령문
```

순서도



설명 if 문의 조건식1의 결과가 True라면 명령문1, ... 을 실행하고, False라면 조건식2를 확인하여 결과가 True라면 명령문2, ... 를 실행하고, False라면 명령문3, ... 을 실행한 뒤 if 문 다음 명령문을 실행한다.

2.8 반복문과 기타 제어문

반복문과 기타 제어문

파이썬 반복문 개요

- 파이썬 반복문 형태인 for 문과 while 문의 공통 및 각각의 규칙은 다음과 같다.
- ① for 문과 while 문의 조건식 끝에는 항상 :(콜론)이 있어야 한다.
- ② 실행 코드는 반드시 공백(스페이스바 또는 Tab)으로 들여쓰기(indent)를 하여 for 문과 while 문에 포함되는(종속) 코드로 작성해야 한다.

표 2-11 for, while 문의 규칙

for 문의 규칙	while 문의 규칙
1. 제어변수와 특정 범위를 활용하여 반복 실행한다. 2. 특정 범위는 range() 함수, 리스트, 문자열 등을 이용하여 지정한다.	1. 선택문처럼 조건식의 결과가 참 또는 거짓인지에 따라 특정 문장을 반복 실행하거나 실행하지 않는 조건적 반복 실행 구조이다. 2. while 문의 조건식들은 논리 연산이 가능한 문장, 즉 참 · 거짓 판별이 가능한 것이어야 한다.

반복문과 기타 제어문

for 문

- for 문은 range() 함수, 리스트, 문자열 등을 이용하여 특정 범위를 지정함
- range() 함수를 이용하는 경우, 시작 값부터 종료 값까지 증가 값만큼 씩 증가시키면서 항목의 원소를 하나씩 꺼내 와 제어변수에 저장한 뒤, for 문에 종속된 명령문들을 실행함
- range() 함수는 range(0, 10, 1)과 range(0, 10)과 range(10)은 모두 0부터 9까지 1씩 증가하며, 숫자를 하나씩 가져와 제어변수에 저장한다는 의미함
- 즉, 시작 값을 생략하면 0이 되고 증가 값을 생략하면 1이 되지만 종료 값은 생략할 수 없으며, 시작 값과 증가 값은 정수형이어야 하고 증가 값은 0이 될 수 없음

반복문과 기타 제어문

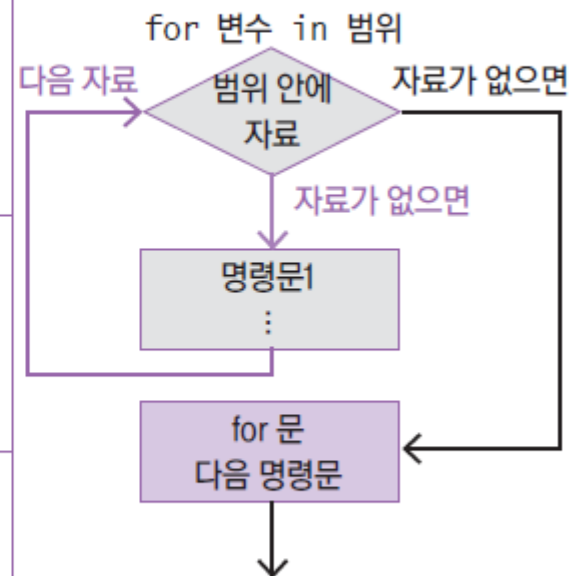
for 문

기본 문법

①
for 제어변수 in range(시작값, 종료값+1, 증가값) :
(공백) 명령문1 ②
⋮
③ 다음 항목
for 문 다음 명령문

for 제어변수 in 리스트 :
(공백) 명령문1
⋮
for 문 다음 명령문

for 제어변수 in '문자열' :
(공백) 명령문1
⋮
for 문 다음 명령문



반복문과 기타 제어문

while 문

- while 문은 선택문처럼 조건식의 결과에 따라 실행 여부가 결정되는데, 선택문은 조건식의 결과가 참이면 실행하고 거짓이면 건너뛰는 과정을 한 번만 실행함
- 반면 while 문은 조건식의 결과가 참이면 종속된 특정 문장을 실행하고, 조건식을 다시 확인하여 결과가 참이면 다시 반복 실행, 거짓이면 종료함
- 즉, 조건 확인과 실행을 계속적으로 반복하는 조건적 반복 실행 구조

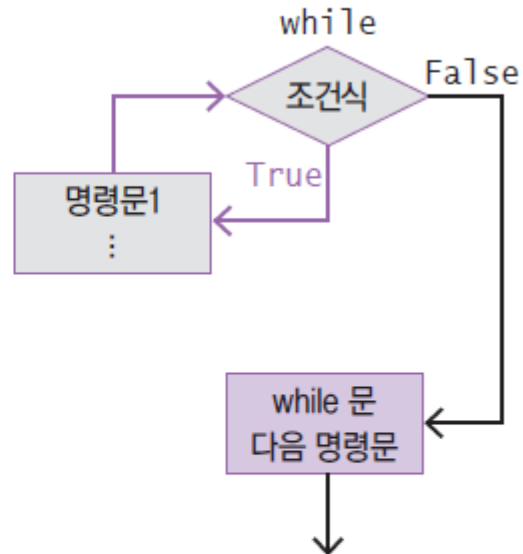
반복문과 기타 제어문

while 문

기본 문법

```
while 조건식:  
    (공백) 명령문1  
    :  
while 문 다음 명령문
```

순서도



설명

while 문의 조건식 결과가 True라면 명령문1, ... 을 실행하고, 이후 while 문의 조건식을 다시 확인하여 결과가 True라면 명령문1, ... 을 실행하는 과정을 반복한다. 그러다가 조건식의 결과가 False가 되면 명령문1, ... 을 건너뛰고 while 문 구조를 벗어나 while 문 다음의 명령문을 실행한다.

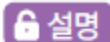
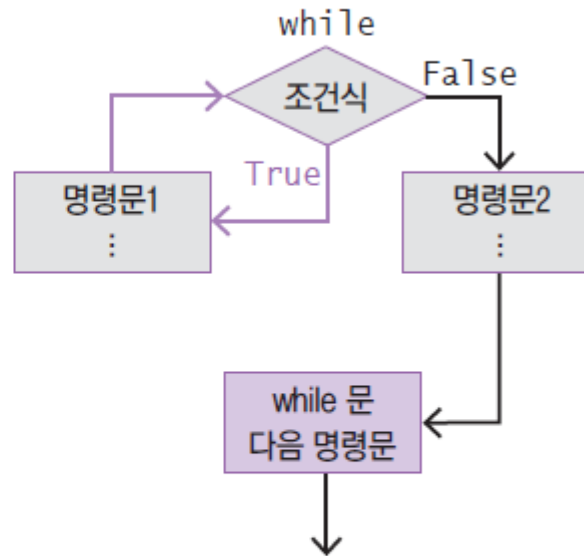
반복문과 기타 제어문

while~else 문

기본 문법

```
while 조건식:  
    (공백) 명령문1  
    :  
else:  
    (공백) 명령문2  
    :  
while 문 다음 명령문
```

순서도



설명

while 문의 조건식 결과가 True라면 명령문1, ... 을 실행하고, 이후 while 문의 조건식을 다시 확인하여 결과가 True라면 명령문1, ... 을 실행하는 과정을 반복한다. 그러다가 조건식의 결과가 False가 되면 명령문2, ... 를 실행하고 while 문 구조를 벗어나 while 문 다음의 명령문을 실행한다.

반복문과 기타 제어문

기타 제어문

- `break` 문을 만나게 되면 반복 구간을 더 이상 실행하지 않고 반복문을 강제 종료함
- `continue` 문을 만나게 되면 반복 구간의 남은 명령문을 실행하지 않고 반복문 처음으로 이동함
- `pass` 문은 아무것도 실행하지 않는 문장으로, 반복문이나 임시 함수 작성 시 기본 틀만 표현하려고 내부에 당장 아무것도 작성하지 않으면 오류가 발생하는데, 이 때 임시적으로 `pass` 문을 작성하여 오류 없이 기본 틀을 갖출 수 있음. 이후 실제 코드를 작성할 때는 `pass` 문을 삭제해야 함

pass 사용 형태

```
1 while True:
2     pass
```

```
1 for x in range(10):
2     pass
```

2.9 함수

함수란?

- 특정한 작업을 위해 서로 관련되어 있는 일련의 명령문으로, 함수를 정의할 때 이름을 지정하고 그 함수가 수행할 일을 위해 순서대로 명령문을 작성하며, 프로그래밍을 하면서 함수의 이름으로 함수를 호출하여 사용함

함수를 사용하는 이유

- 첫째, 중복된 코드를 별도로 분류하여 함수로 만들게 되면 코드의 양을 줄일 수 있고 이로 인해서 가독성이 좋아진다.
- 둘째, 간결해진 코드는 전체의 기능을 이해하기에 수월하다.
- 셋째, 프로그램의 흐름을 파악하기 쉽기 때문에 관리하기도 쉽다.
- 넷째, 중요한 기능만 함수로 정의해두면 이를 필요로 하는 프로그램은 물론 다른 프로그램에도 재사용이 가능하여 매우 효율적이다.

함수의 종류

- 내장 함수와 사용자 정의 함수 두 가지로 나눌 수 있는데, 프로그램에 이미 지정된 함수를 내장 함수라고 하고 사용자 정의 함수는 사용자가 필요에 의해서 직접 정의하는 함수를 말함

주요 내장 함수

내장 함수	설명	사용법
abs()	숫자의 절댓값을 반환하는 함수	abs(값)
chr()	유니코드(unicode) 값을 입력받아 그 코드에 해당하는 문자를 출력하는 함수	chr(유니코드 값)
enumerate()	컬렉션 자료형을 입력받아 인덱스 값을 포함하는 enumerate 객체로 반환하는 함수	enumerate(컬렉션 자료형)
len()	값의 길이(요소의 전체 개수)를 반환하는 함수	len(반복 가능 객체)
list()	리스트로 만들어 반환하는 함수	list(반복 가능 객체)
max()	최댓값을 반환하는 함수	max(반복 가능 객체)
min()	최솟값을 반환하는 함수	min(반복 가능 객체)

주요 내장 함수

내장 함수	설명	사용법
ord()	문자의 유니코드 값을 반환하는 함수	ord(문자)
pow()	제곱 값을 반환하는 함수	pow(값1, 값2)
round()	반올림하는 함수	round(값, [소수점 자리])
str()	문자열로 변환하여 반환하는 함수	str(값)
tuple()	튜플로 만들어 반환하는 함수	tuple(반복 가능 객체)
type()	자료형을 알려주는 함수	type(값)
zip()	동일한 개수로 이루어진 자료형을 묶어 주는 함수	zip(반복 가능 객체)

enumerate() 함수

enumerate() 사용 예

```
1 for x, name in enumerate(['수박', '참외', '자두']):  
2     print(x, name)
```

```
0 수박  
1 참외  
2 자두
```

zip() 함수

zip() 사용 예 1

```
1 num = [1, 2, 3]  
2 char = ['A', 'B', 'C']  
3 for item in zip(num, char):  
4     print(item)
```

```
(1, 'A')  
(2, 'B')  
(3, 'C')
```

zip() 사용 예 2

```
1 num = [1, 2, 3]  
2 char = ['A', 'B', 'C']  
3 dict(zip(num, char))
```

```
1: 'A', 2: 'B', 3: 'C'
```

난수 생성 함수

☰ 랜덤 모듈을 이용하여 난수를 생성하는 예

1	import random	0.11132867921152356
2		0.5950949227890241
3	for i in range(10):	0.04820265884996877
4	x = random.random()	0.841003109276478
5	print(x)	0.997914947094958
		0.04842330803368111
		0.7416295948208405
		0.510535245390327
		0.27447040171978143
		0.028511805472785867

☰ randint() 사용 예

1	import random	1	random.randint(5, 10)
2	random.randint(5, 10)		
6			10

난수 생성 함수

choice() 사용 예

```
1 import random
2 num = [1, 2, 3]
3 random.choice(num)
```

1

```
1 random.choice(num)
```

3

사용자 정의 함수

- 사용자 정의 함수를 선언할 때는 def라는 예약어를 사용함
- def는 '정의하다'라는 뜻으로 영어 단어 'define'의 앞 글자에서 딴 것으로, 함수 이름을 작성할 때에도 변수에서 이름을 정의했던 것과 같이 규칙을 따라야 함
 - 알파벳, 숫자, 밑줄 문자(_)만 사용 가능
 - 함수의 첫 글자는 반드시 알파벳 또는 밑줄 문자만 사용 가능
 - 공백 사용 불가
 - 예약어(print, def 등) 사용 불가
- 함수 이름 뒤에는 빈 괄호를 넣거나, 괄호 안에 인수를 입력받는 함수로 작성할 수 있는데, 이때, 인수(argument)는 호출된 함수에 전달할 값이고 매개변수(parameter)는 호출된 함수에서 전달받은 값을 임시로 할당하는 변수임
- 매개변수로부터 처리된 작업의 결과는 반환값(return value)이고 호출한 함수로 값이 반환되는데, 이때 반환되는 값은 반드시 하나이고 인수, 매개변수, 반환값은 항상 존재해야 하는 것은 아님

사용자 정의 함수 기본 구조

함수의 기본 구조

```
def 함수 이름(매개변수1, 매개변수2, ...) :  
    문장1  
    ...  
    return 반환값
```

사용자 정의 함수 사용 예

두 개의 수를 입력받아 큰 수를 결정하는 함수의 예

```
1 def max1(num1,num2):  
2     if num1>num2:  
3         return num1  
4     else:  
5         return num2  
6 x = int(input('정수1: '))  
7 y = int(input('정수2: '))  
8 print('큰 수:',max1(x,y))
```

실행 결과

```
정수1: 35  
정수2: 25  
큰 수: 35
```

람다(lambda) 함수

- 이름이 없는 익명 함수를 람다(lambda) 함수라고 함
- 람다 함수의 표현식은 식 형태로 되어 있어 람다 표현식(lambda expression)이라고 부르며, 람다 표현식은 함수를 간편하게 작성할 수 있어 다른 함수의 인수로 넣을 때 많이 사용함

람다 표현식 작성

☐ 람다 함수의 기본 구조

람다 매개변수들: 식

람다 표현식 작성

- 사용자 정의 함수로 표현했을 때

```
1 def plus_five(x):  
2     return x + 5  
3  
4 plus_five(20)
```

실행 결과

25

- 람다 함수로 표현했을 때

☞ 람다 함수 사용 예 1

```
1 lambda x: x + 5
```

<function __main__.<lambda>>



☞ 람다 함수 사용 예 2

```
1 plus_five = lambda x: x + 5  
2 plus_five(20)
```

실행 결과

25

람다 표현식을 인수로 사용

map() 함수 사용 예

```
1 def plus_five(x):  
2     return x + 5  
3  
4 list(map(plus_five, [15, 25, 35]))
```

실행 결과

[20, 30, 40]

람다 표현식을 인수로 사용한 예

```
1 list(map(lambda x: x + 5, [15, 25, 35]))
```

실행 결과

[20, 30, 40]

수고하셨습니다.

